

GPUMODE Lecture Study Notes: Lecture 67

NCCL and NVSHMEM

Core Problem the Lecture Addresses

This lecture explains why NVIDIA maintains two major GPU communication libraries, **NCCL** and **NVSHMEM**, and how their programming models map to modern distributed GPU workloads. The central problem is not merely “how do GPUs communicate?” but rather:

How do we express communication so that GPU computation, memory movement, synchronization, and network

Modern deep learning systems use many communication patterns:

- **Data parallelism:** gradients are reduced across replicas, often using all-reduce.
- **Tensor parallelism:** individual layers are partitioned, often using all-gather, reduce-scatter, or all-to-all.
- **Pipeline parallelism:** activations and gradients move between stages.
- **Expert parallelism:** tokens are dynamically routed to experts, creating irregular all-to-all communication.
- **Inference serving:** latency is often more important than bulk bandwidth, especially when routing small or medium payloads.

The lecture contrasts two broad models:

- **NCCL:** a communication library optimized around collective and point-to-point communication for GPU workloads. It inherits many ideas from MPI, but intentionally removes some MPI generality to make GPU communication fast and practical.
- **NVSHMEM:** a partitioned global address space model that exposes one-sided communication, symmetric memory, remote load/store, and device-initiated communication from inside CUDA kernels.

The main engineering trade-off is:

Generality and synchronization convenience versus low-overhead, explicit, fine-grained control.

MPI-style two-sided communication is convenient and portable, but it often combines **data movement** and **synchronization**. NVSHMEM-style one-sided communication separates these concerns. This separation can eliminate unnecessary handshakes and enable fused communication-computation kernels, but the programmer must manage ordering and synchronization explicitly.

Required Background

Distributed GPU Execution

A distributed GPU program usually consists of multiple communication endpoints. In MPI these are called **ranks**. In OpenSHMEM and NVSHMEM they are often called **processing elements**, abbreviated as **PEs**. In NCCL, the analogous concept is a NCCL rank inside a communicator.

A typical multi-GPU program has:

- one process per GPU,
- one rank or PE per GPU,
- a communicator or team describing which GPUs participate in communication,
- CUDA streams controlling ordering between compute kernels and communication operations.

For GPU communication, the basic performance goal is to keep the GPU busy while data moves through the fastest possible path:

GPU compute + HBM + NVLink or network + copy engines or NICs.

GPU Memory Hierarchy

A GPU program touches several memory spaces:

- **Registers**: private per thread, fastest, limited capacity.
- **Shared memory**: explicitly managed memory shared within a thread block.
- **L1 cache**: close to an SM, architecture-dependent behavior.
- **L2 cache**: shared across the GPU.

- **HBM**: high-bandwidth global memory local to a GPU.
- **Peer GPU memory**: memory physically attached to another GPU, reachable through NVLink or PCIe if supported.
- **Remote node memory**: GPU memory on another node, reachable through networking such as InfiniBand, Ethernet, AWS EFA, or Slingshot.

The approximate bandwidth hierarchy discussed in the lecture is:

$$\text{HBM} \gg \text{NVLink} \gg \text{PCIe or inter-node network}.$$

On a DGX H100-like system, representative useful numbers are:

$$\begin{aligned} B_{\text{HBM, aggregate}} &\approx 24 \text{ TB/s for 8 GPUs,} \\ B_{\text{NVLink switch bisection}} &\approx 3 \text{ TB/s,} \\ B_{\text{PCIe Gen5}} &\approx 50 \text{ GB/s in a favorable case.} \end{aligned}$$

The exact values depend on hardware and topology, but the important intuition is that each step away from local HBM can cost nearly an order of magnitude of bandwidth.

CUDA Streams and Communication Ordering

CUDA streams are command queues. Kernels and communication operations inserted into the same stream execute in program order. This matters because CPU-oriented communication APIs often require the user to manually synchronize before launching communication.

A host-side pattern with MPI may look like:

```
kernel<<<grid, block, 0, stream>>>(A, B);
cudaStreamSynchronize(stream);
MPI_Send(B, count, MPI_FLOAT, peer, tag, MPI_COMM_WORLD);
```

The synchronization forces the host to wait until the GPU work finishes before MPI observes the buffer. In contrast, NCCL and NVSHMEM stream APIs can enqueue communication directly after GPU kernels:

```
kernel<<<grid, block, 0, stream>>>(A, B);
ncclAllReduce(B, C, count, ncclFloat, ncclSum, comm, stream);
```

The CUDA runtime and NCCL coordinate stream ordering without requiring the host to block.

Two-Sided Versus One-Sided Communication

Two-Sided Communication

Two-sided communication means both sender and receiver participate. MPI send/receive is the canonical model.

The sender knows:

(source buffer, destination rank, tag, datatype, count).

The receiver knows:

(target buffer, source rank or wildcard, tag, datatype, count).

A matching protocol connects the two halves. A simplified rendezvous protocol is:

1. Sender announces intent to send.
2. Receiver posts a matching receive.
3. Receiver exposes the destination buffer, possibly registering memory with the network.
4. Sender receives permission or a remote key.
5. Sender performs the data transfer, often using RDMA write.

This gives useful ordering and synchronization semantics, but it introduces metadata exchange and potential waiting.

One-Sided Communication

One-sided communication separates the active side from the passive side. A `put` writes to remote memory. A `get` reads from remote memory. The remote PE does not need to post a matching receive at that moment.

A one-sided put conceptually performs:

$$\text{remote memory}[\text{offset}] \leftarrow \text{local memory}[\text{offset}].$$

A one-sided get conceptually performs:

$$\text{local memory}[\text{offset}] \leftarrow \text{remote memory}[\text{offset}].$$

The active side must already know that the target memory is accessible. This is usually established through a prior collective allocation or memory registration step.

Remote Direct Memory Access

RDMA allows a NIC or interconnect to read or write remote memory without involving the remote CPU in the critical path. For GPU communication, GPUDirect RDMA and similar mechanisms allow network devices to interact with GPU memory directly.

The idealized communication cost can be modeled as:

$$T(n) = \alpha + \frac{n}{B},$$

where:

- $T(n)$ is communication time for n bytes,
- α is latency or fixed overhead,
- B is sustained bandwidth.

For many small messages, α dominates. For very large messages, n/B dominates. This is why fine-grained communication can perform poorly unless the runtime or programmer aggregates work.

Main Concepts

Why NCCL Exists

NCCL exists because MPI is powerful and portable, but much of its generality is expensive or awkward for GPU-centric communication. MPI was designed in the early 1990s for CPU-oriented distributed memory systems. Its strengths include:

- portability across nearly all computing environments,
- first-class collectives such as all-reduce, broadcast, reduce-scatter, and all-to-all,
- robust send/receive semantics,
- long-term backwards compatibility,
- powerful datatypes and message matching.

However, several MPI features are problematic for efficient GPU execution:

- **Datatype generality:** MPI datatypes can represent deeply nested, non-contiguous memory layouts. This is elegant for CPU code but undesirable to execute inside GPU kernels.
- **Tags:** messages must be distinguished by tags, adding matching logic.

- **Wildcards:** `MPI_ANY_SOURCE` and similar features make matching less local and harder to accelerate.
- **Underflow:** receive buffers may be larger than sent messages, meaning receiver-side protocol decisions cannot always be made independently.
- **Multiple ranks per GPU:** useful in MPI applications, but it complicates GPU communication runtime logic.
- **Host-oriented progress:** MPI operations often involve host synchronization unless special CUDA-aware paths and stream integration exist.

NCCL intentionally avoids much of this complexity. It focuses on the dominant communication patterns needed by AI workloads and exposes them in a CUDA-stream-friendly way.

NCCL's Design Center

NCCL is designed for:

- GPU collectives,
- stream-ordered communication,
- multi-GPU single-node and multi-node execution,
- high-performance all-reduce, all-gather, reduce-scatter, broadcast, send, receive, and related patterns,
- common AI parallelism strategies.

A key design principle is:

Expose high-level communication intent and let the library choose the algorithm and protocol.

For example, an all-reduce of 8 bytes should not use the same strategy as an all-reduce of 1 GB. The runtime should choose among tree, ring, multi-ring, hierarchical, or topology-aware algorithms.

A simplified decision model is:

$\text{algorithm} = f(\text{message size, topology, GPU count, network, datatype, operation}).$

The user should generally not manually tune NCCL protocols unless there is strong profiling evidence. Otherwise, a setting that helps one machine or batch size may hurt another.

Why NVSHMEM Exists

NVSHMEM is based on the SHMEM family of one-sided programming models. Its central abstraction is a **partitioned global address space**, abbreviated **PGAS**.

The memory model is:

$$\text{global view} = \bigcup_{p=0}^{P-1} \text{symmetric heap}_p,$$

where P is the number of PEs. Each PE contributes a corresponding memory region called the **symmetric heap**. Objects allocated collectively in this heap can be addressed by other PEs.

This gives programmers a way to write:

put data into PE q , get data from PE q , or access PE q 's memory through a pointer if possible.

NVSHMEM is especially important because it supports:

- host-initiated one-sided communication,
- stream-ordered communication,
- **device-initiated communication from inside CUDA kernels**,
- warp-level and block-level cooperative communication,
- remote pointer access within NVLink domains,
- collectives over symmetric memory,
- low-latency and zero-SM communication paths in some cases.

Device-Initiated Communication

The distinctive NVSHMEM feature emphasized in the lecture is that communication can be initiated directly from GPU code.

A conceptual kernel can do:

```
__global__ void route_tokens(float* local_tokens,
                             float* remote_mailbox,
                             int* destinations) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int dst_pe = destinations[tid];

    // Conceptual one-sided put from device code.
```

```

    nvshmem_float_p(remote_mailbox + tid, local_tokens[tid], dst_pe);
}

```

The important hardware implication is that the GPU does not need to return to the host between compute and communication. This removes kernel launch and host scheduling from the critical path.

If a workload repeatedly performs:

compute small routing decision → send small payload → wait for another device decision,

then host-driven communication can be too coarse. Device-initiated communication enables much finer interleaving:

GPU compute → GPU initiated communication → GPU synchronization → more GPU compute.

Thread, Warp, and Block Granularity

NVSHMEM exposes communication at multiple granularities:

- **thread-level:** a single GPU thread initiates communication;
- **warp-level:** a warp cooperatively performs communication;
- **block-level:** a CUDA thread block cooperatively performs communication.

The reason is memory coalescing and amortization. A thread-level remote put of one element can be latency dominated. A warp-level or block-level operation can aggregate many lanes and generate more efficient memory transactions.

A simplified efficiency model is:

$$\eta(n) = \frac{n/B}{\alpha + n/B},$$

where $\eta(n)$ is transfer efficiency for n bytes. For small n , $\eta(n)$ is low. Increasing the cooperative transfer size improves amortization of α .

NCCL Versus NVSHMEM at a Glance

Dimension	NCCL	NVSHMEM
Primary model	Collectives and point-to-point	One-sided PGAS plus collectives
Historical influence	MPI collectives and GPU communication	SHMEM and remote memory access

Dimension	NCCL	NVSHMEM
Synchronization style	Often implicit in collective or send/receive semantics	Explicit; data movement and synchronization are separated
CUDA stream integration	Strong	Strong
Device-initiated communication	Emerging through roadmap and previews	Core capability today
Remote pointer access	Emerging through symmetric memory features	Supported within reachable domains such as NVLink
Best fit	Standard AI distributed training and framework integration	Fine-grained, irregular, fused communication-computation kernels

Historical Context

MPI

MPI began in the early 1990s as a standardization effort. Its design goal was not to invent a completely new model, but to unify many existing vendor-specific and research communication APIs.

The main reason MPI succeeded is:

portability + stable semantics + useful primitives.

The lecture emphasizes that MPI's portability was a major achievement. It could map onto supercomputer networks, Ethernet, sockets, and many other environments. This made MPI the durable lingua franca of HPC.

MPI also got collectives right. Instead of forcing users to manually implement all-reduce or broadcast, MPI exposes high-level communication intent:

```
MPI_Allreduce(sendbuf, recvbuf, count, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
```

The implementation can then choose algorithms based on message size and topology.

SHMEM

SHMEM originated in the context of Cray systems such as the T3D. These machines exposed extremely fast remote memory operations. The SHMEM philosophy was:

If the hardware can make remote memory access fast, expose a lean API that lets programmers use it directly.

SHMEM’s key idea is one-sided communication:

```
shmem_putmem(remote_target, local_source, nbytes, pe);
```

This resembles `memcpy`, except it writes into another PE’s memory. The target PE does not post a receive.

OpenSHMEM later standardized this style. NVSHMEM adapts it to NVIDIA GPUs, CUDA streams, NVLink, and GPU-initiated communication.

Structured Versus Unstructured Communication

MPI is excellent for many structured workloads, such as stencil computations. In a domain decomposition, each rank often knows exactly which neighbors it communicates with:

$$\text{rank } r \leftrightarrow \{r_{\text{left}}, r_{\text{right}}, r_{\text{up}}, r_{\text{down}}\}.$$

The communication graph is regular.

SHMEM-style models shine in irregular workloads, where the sender knows what it wants to send but the receiver may not know what is coming. Examples include:

- distributed sorting,
- graph analytics,
- irregular sparse computations,
- mixture-of-experts token routing,
- mailbox-style producer-consumer patterns.

For irregular routing, the active side may simply write into remote memory:

$$\text{mailbox}_{\text{expert}}[\text{slot}] \leftarrow \text{token}.$$

The receiver later inspects its mailbox.

Hardware-Level Explanation

The Communication Path

A GPU communication operation can move through several possible paths:

- **local HBM access:** fastest path, handled by GPU memory controllers;
- **NVLink peer access:** remote GPU memory accessed through NVLink or an NVLink switch;

- **PCIe peer access:** slower peer path when NVLink is unavailable;
- **network RDMA:** inter-node communication through NICs;
- **host-mediated path:** undesirable for performance-critical GPU communication.

A simplified bandwidth roofline for communication is:

$$T_{\text{total}} = T_{\text{local compute}} + T_{\text{local memory}} + T_{\text{peer or network communication}} + T_{\text{synchronization}}.$$

Optimizing distributed GPU workloads usually means reducing one or more of:

$$T_{\text{peer or network communication}}, \quad T_{\text{synchronization}}, \quad T_{\text{host overhead}}, \quad T_{\text{idle GPU time}}.$$

NVLink as a Load/Store Fabric

Within an NVLink domain, remote GPU memory can sometimes be accessed as if it were local from the program's perspective. NVSHMEM exposes this through remote pointer lookup.

The programmer asks:

Can PE p 's address be mapped into my address space as a usable pointer?

If yes, the program can use a pointer directly. If no, it falls back to explicit put/get.

Conceptually:

```
void* ptr = nvshmem_ptr(remote_object, remote_pe);

if (ptr != nullptr) {
    // Direct load/store through NVLink-accessible pointer.
    use_as_pointer(ptr);
} else {
    // Fall back to explicit get or put.
    nvshmem_getmem(local_buffer, remote_object, nbytes, remote_pe);
}
```

This creates a hierarchy-aware strategy:

if NVLink reachable $\Rightarrow$ load/store pointer access,

otherwise $\Rightarrow$ explicit communication through network path.

Why Remote Pointer Access Can Be Faster

Explicit communication separates the operation into phases:

communicate block $\rightarrow$ locally compute on block $\rightarrow$ synchronize.

Remote pointer access can fuse the memory access into the compute kernel:

compute kernel directly loads remote data as needed.

This can reduce:

- kernel launch count,
- explicit communication calls,
- intermediate buffering,
- host involvement,
- software overhead,
- artificial phase boundaries between communication and compute.

The lecture's matrix transpose experiment showed that direct pointer access over NVLink improved throughput by approximately 30% relative to more explicit all-to-all styles.

Copy Engines and Zero-SM Communication

Some communication operations can be handled by copy engines or network hardware rather than SMs. This is valuable because SMs should ideally execute model compute:

SM cycles used for communication $\Rightarrow$ fewer cycles for matmul, attention, or expert compute.

A zero-SM communication path attempts to move data without consuming significant SM resources. This is particularly useful when communication and compute overlap.

Forward Progress and Cooperative Groups

Device-initiated collectives are subtle because collectives imply synchronization. If only some blocks are resident and they wait for blocks that have not yet launched, deadlock is possible. Therefore, device-side collectives require careful launch semantics, such as cooperative group launch, to guarantee forward progress.

The hazard is:

resident blocks wait $\wedge$ nonresident blocks needed for progress $\Rightarrow$ deadlock.

The correct mental model is:

Device-side collectives are powerful, but the programmer must ensure that all participating GPU work has a valid progress guarantee.

Mathematical Reasoning

Communication Time Model

For a message of n bytes:

$$T(n) = \alpha + \frac{n}{B},$$

where:

- α is fixed latency,
- B is bandwidth,
- n/B is serialization time.

For m small messages:

$$T_{\text{many small}} = m\alpha + \sum_{i=1}^m \frac{n_i}{B}.$$

For one aggregated message with the same total bytes:

$$T_{\text{aggregated}} = \alpha + \frac{\sum_{i=1}^m n_i}{B}.$$

The difference is:

$$T_{\text{many small}} - T_{\text{aggregated}} = (m - 1)\alpha.$$

This explains why thread-level elementwise remote gets are usually poor unless the access pattern is truly fine-grained and cannot be aggregated.

Bandwidth Composition

If an operation has a local phase and a communication phase, total time is:

$$T_{\text{op}} = T_{\text{local}} + T_{\text{comm}}.$$

If the amount of data processed is D , the effective bandwidth is:

$$B_{\text{eff}} = \frac{D}{T_{\text{local}} + T_{\text{comm}}}.$$

If local GPU memory bandwidth is high but communication bandwidth is lower, communication dominates:

$$T_{\text{comm}} = \frac{D_{\text{comm}}}{B_{\text{comm}}}, \quad B_{\text{comm}} \ll B_{\text{HBM}}.$$

Therefore even if local transpose reaches approximately 23 to 24 TB/s, an all-to-all phase through an NVLink switch at roughly 2.7 to 3 TB/s can dominate the end-to-end result.

Matrix Transpose as an All-to-All

Consider a matrix decomposed by block columns across P GPUs. GPU p owns block column p . A global transpose turns block columns into block rows.

Let the matrix be divided into $P \times P$ tiles:

$$A = \begin{bmatrix} A_{00} & A_{01} & \cdots & A_{0,P-1} \\ A_{10} & A_{11} & \cdots & A_{1,P-1} \\ \vdots & \vdots & \ddots & \vdots \\ A_{P-1,0} & A_{P-1,1} & \cdots & A_{P-1,P-1} \end{bmatrix}.$$

If GPU j owns column j , then it owns:

$$\{A_{0j}, A_{1j}, \dots, A_{P-1,j}\}.$$

After transpose, tile A_{ij} contributes to position:

$$A_{ij}^T \rightarrow B_{ji}.$$

Thus every GPU sends a tile to every other GPU. This is an all-to-all pattern:

$$\forall i, j : \quad \text{GPU } j \rightarrow \text{GPU } i.$$

Accumulating Transpose

The benchmark uses an accumulating transpose:

$$B_{ji} \leftarrow B_{ji} + A_{ij}^T.$$

It also increments A after each iteration:

$$A_{ij} \leftarrow A_{ij} + 1.$$

The increment prevents accidental correctness from applying transpose twice and returning to the original layout. If an implementation makes an invertible mistake, a simple transpose-only verification may not catch it. The update makes elementwise verification stricter.

Expected Performance on DGX H100

The lecture uses rough speed-of-light estimates:

$$B_{\text{local}} \approx 24 \text{ TB/s}, \quad B_{\text{NVLink switch}} \approx 3 \text{ TB/s}.$$

If a method performs local work at B_{local} and communication at $B_{\text{NVLink switch}}$, the effective result should fall between those values:

$$3 \text{ TB/s} < B_{\text{eff}} < 24 \text{ TB/s}.$$

The observed NCCL all-to-all-style implementation achieved roughly:

$$B_{\text{eff}} \approx 7.5 \text{ TB/s}.$$

Breaking down the phases:

$$B_{\text{all-to-all}} \approx 2.7 \text{ TB/s}, \quad B_{\text{local}} \approx 23 \text{ TB/s}.$$

This matches the expected bottleneck: network or NVLink-switch communication is much slower than local HBM bandwidth.

Code Walkthrough

MPI-Style All-to-All Transpose

The MPI-like version has three phases:

1. all-to-all exchange tiles,

2. locally transpose received tiles,
3. increment the input matrix.

Representative pseudocode:

```
for (int iter = 0; iter < iterations; ++iter) {
    alltoall(A_column_blocks, T_row_blocks, block_bytes);

    for (int block = 0; block < nranks; ++block) {
        local_transpose(T_row_blocks[block], B_row_blocks[block]);
    }

    increment_matrix(A_column_blocks);
}
```

The important performance issue is phase separation. First, all GPUs flood the communication fabric. Then all GPUs perform local transpose. Then all GPUs perform increment. This is simple and lets the library optimize all-to-all, but it may not overlap communication with useful local work.

Point-to-Point Pairwise Exchange

A user-managed point-to-point version can exchange one partner at a time:

```
for (int step = 0; step < nranks; ++step) {
    int peer = (rank + step) % nranks;

    sendrecv(A_tile_for_peer,
             recv_tile_from_peer,
             peer);

    local_transpose(recv_tile_from_peer, B_tile_for_peer);
}

increment_matrix(A_column_blocks);
```

The intended idea is flow control. Instead of injecting all messages into the network at once, each rank communicates with one peer and then performs local work.

The benefit could be:

less network queue pressure+some interleaving of communication and computation.

However, the lecture shows that this was a poor match for NCCL's send/receive behavior. NCCL send/receive is designed for building custom collectives and may

already include scheduling or throttling assumptions. If the user adds another layer of manual throttling, the result can be worse.

The practical lesson is:

Do not outsmart NCCL unless profiling proves that the default collective or grouped send/receive pattern is wrong.

NCCL Grouped Send/Receive All-to-All

NCCL did not historically expose every possible collective as a single direct call in all contexts, but all-to-all can be built from grouped sends and receives.

Representative structure:

```
ncclGroupStart();

for (int peer = 0; peer < nranks; ++peer) {
    ncclSend(sendbuf_for_peer,
             count,
             ncclFloat,
             peer,
             comm,
             stream);

    ncclRecv(recvbuf_from_peer,
             count,
             ncclFloat,
             peer,
             comm,
             stream);
}

ncclGroupEnd();
```

The group tells NCCL that these operations are part of a larger communication pattern. NCCL can schedule them together.

Hardware implications:

- The communication is enqueued on a CUDA stream.
- The CPU does not need to synchronize after every GPU operation.
- NCCL can select protocols and scheduling internally.
- The operation remains explicit communication rather than direct load/store.

NVSHMEM Put

A host-side NVSHMEM put resembles remote memcpy:

```
nvshmem_putmem(remote_target,  
               local_source,  
               nbytes,  
               target_pe);
```

The active PE writes into the target PE's symmetric heap. The target PE does not post a matching receive.

This is useful for mailbox-style algorithms:

```
int slot = reserve_remote_slot(target_pe);  
nvshmem_putmem(remote_mailbox + slot,  
               local_payload,  
               payload_bytes,  
               target_pe);
```

The missing detail is synchronization. The target must know when data is valid. This is commonly handled using notification, counters, flags, or barriers.

NVSHMEM Get

A get pulls from remote memory:

```
nvshmem_getmem(local_buffer,  
               remote_source,  
               nbytes,  
               source_pe);
```

In the transpose benchmark, a get-based implementation can fetch the tile needed from each peer and then locally transpose it.

Representative pattern:

```
for (int peer = 0; peer < npes; ++peer) {  
    nvshmem_getmem(local_tile,  
                   remote_tile_address(peer),  
                   tile_bytes,  
                   peer);  
  
    local_transpose(local_tile, output_tile);  
}  
  
nvshmem_barrier_all();  
increment_matrix(A);  
nvshmem_barrier_all();
```

The two barriers are needed because one-sided get does not automatically synchronize with the remote update to A . The program must ensure:

all reads of A complete before any PE updates A ,

and:

all updates to A complete before the next iteration reads A .

Device-Side Elementwise Get

A naive device-side version might replace a local load with an NVSHMEM get-like operation inside the kernel.

Representative conceptual code:

```
__global__ void transpose_remote_elementwise(float* B,
                                              float* A_remote_base,
                                              int source_pe,
                                              int n) {
    int i = blockIdx.y * blockDim.y + threadIdx.y;
    int j = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n && j < n) {
        float value;
        nvshmem_float_get(&value,
                          A_remote_base + j * n + i,
                          1,
                          source_pe);
        B[i * n + j] += value;
    }
}
```

The lecture explicitly warns that this is usually not the best pattern. It may work within an NVLink domain because remote loads can map efficiently, but as a general network pattern it is too fine-grained.

The problem is:

one remote transaction per element $\Rightarrow$ poor latency amortization.

A better version would use warp-level or block-level cooperative get, or remote pointer access.

NVSHMEM Remote Pointer Access

The more interesting implementation uses `nvshmem_ptr` to obtain a direct pointer to remote symmetric memory when the peer is reachable through a load/store-capable fabric.

Representative code:

```
float* remote_ptr =
    static_cast<float*>(nvshmem_ptr(remote_tile, peer));

if (remote_ptr != nullptr) {
    transpose_kernel<<<grid, block, 0, stream>>>(remote_ptr,
                                                  local_output,
                                                  tile_shape);
} else {
    nvshmem_getmem(local_temp,
                  remote_tile,
                  tile_bytes,
                  peer);
    transpose_kernel<<<grid, block, 0, stream>>>(local_temp,
                                                  local_output,
                                                  tile_shape);
}
```

This is the hierarchy-aware strategy:

1. Try to treat remote memory as directly accessible.
2. If pointer access is valid, let the GPU load remote data through NVLink.
3. Otherwise, use explicit get/put over the network.

Inside an NVLink domain, this can remove explicit communication calls from the core algorithm. The compute kernel naturally performs loads from remote GPU memory.

Bulk Kernel Launch Optimization

The lecture mentions that a non-bulk version launched many small CUDA kernels, while a bulk version pushed more work into a single kernel using an additional grid dimension.

A non-bulk pattern:

```
for (int tile = 0; tile < ntiles; ++tile) {
    transpose_tile_kernel<<<grid, block, 0, stream>>>(tile_input(tile),
                                                       tile_output(tile));
}
```

A bulk pattern:

```
dim3 grid(blocks_x, blocks_y, ntiles);
transpose_many_tiles_kernel<<<grid, block, 0, stream>>>(input,
                                                         output,
                                                         ntiles);
```

The bulk kernel maps:

blockIdx.z $\rightarrow$ tile index.

This reduces launch overhead and gives the GPU scheduler more work per launch.

Performance Intuition

Why NCCL All-to-All Behaves Predictably

NCCL all-to-all-style grouped communication saturates the communication fabric and then local kernels saturate HBM. The measured phase behavior was close to the expected limits:

$$B_{\text{communication}} \approx 2.7 \text{ TB/s}, \quad B_{\text{local}} \approx 23 \text{ TB/s}.$$

This is exactly what a roofline-style model predicts. The all-to-all phase is communication-bound. The local transpose phase is memory-bandwidth-bound.

Why Manual Pairwise NCCL Was Slower

The manual pairwise implementation attempted to improve performance by throttling communication. However, NCCL already has internal assumptions for grouped send/receive scheduling. The manual pairwise logic interfered with NCCL’s own scheduling.

The failure mode is:

user throttling + library throttling $\rightarrow$ underutilized network.

The result was stair-step performance and lower throughput.

Practical rule:

Use NCCL collectives or grouped communication naturally first. Only manually schedule communication after profiling and understanding NCCL’s protocol behavior.

Why NVSHMEM All-to-All Looks Similar to NCCL

NVSHMEM all-to-all can be implemented using one-sided puts. For large enough messages, the data path is similar: move many bytes across the same NVLink or network fabric.

Thus asymptotically:

$$B_{\text{NVSHMEM all-to-all}} \approx B_{\text{NCCL all-to-all}}$$

for large bulk transfers on the same topology. Differences are more visible for small messages, where latency and synchronization semantics matter.

Why Remote Pointer Access Improved Performance

Remote pointer access removes a phase boundary. Instead of:

fetch tile $\rightarrow$ transpose tile $\rightarrow$ repeat,

the kernel can:

load remote tile elements $\rightarrow$ transpose into local output

inside the same compute operation.

This improves performance because:

- fewer communication API calls are needed,
- fewer intermediate buffers may be required,
- memory movement is demand-driven by the compute kernel,
- the GPU's memory system and NVLink fabric handle remote loads,
- communication and computation are naturally fused.

The lecture reported approximately:

$$B_{\text{explicit}} \approx 7.5 \text{ TB/s}, \quad B_{\text{remote pointer}} \approx 9.5 \text{ TB/s}.$$

The relative improvement is:

$$\frac{9.5 - 7.5}{7.5} \approx 0.267,$$

which is approximately 27%, described as roughly a 30% improvement.

Why Small Messages Are Dangerous

Small messages suffer from latency and poor network injection efficiency. If each GPU thread sends one element, the system may generate many tiny operations:

$$m \text{ messages of 4 bytes} \Rightarrow m\alpha \text{ overhead terms.}$$

Better approaches include:

- coalescing into warp-level communication,
- coalescing into block-level communication,
- using shared memory to stage contiguous chunks,
- using remote pointer access within NVLink domains,
- using mailbox buffers and batched notifications.

Structured Halo Exchange

For stencil halo exchange, one halo direction may be contiguous and another may be strided. A strided halo can create inefficient small or non-contiguous communication.

A remote pointer model can help because the GPU can directly load strided remote data through NVLink, allowing normal GPU memory coalescing and latency hiding mechanisms to operate. However, this is topology-dependent: remote pointer access over NVLink is very different from fine-grained get over a multi-node network.

Expert Parallelism and Mixture-of-Experts

The Communication Pattern

In a mixture-of-experts layer, a router assigns tokens to experts. The routing decision is dynamic:

$$\text{token } t \rightarrow \text{expert } e(t).$$

If experts are distributed across GPUs, tokens must be sent to the GPUs owning the selected experts.

The pattern is an irregular all-to-all:

$$\forall \text{ GPU } p, \quad \text{send variable number of tokens to many GPUs.}$$

Unlike dense tensor parallelism, the receive sizes are not necessarily known symmetrically in advance.

Why One-Sided Communication Fits Expert Routing

The router knows where each token should go. The destination expert does not necessarily know which tokens will arrive until routing is complete.

This naturally matches a one-sided put:

$$\text{remote expert mailbox}[k] \leftarrow \text{token payload}.$$

A notification or counter can indicate arrival:

$$\text{counter}_{\text{expert}} \leftarrow \text{counter}_{\text{expert}} + 1.$$

This is often called put-with-notification or mailbox communication.

Dispatch and Combine

Expert parallelism often has two communication phases:

1. **Dispatch**: route tokens from their original GPUs to expert GPUs.
2. **Combine**: send expert outputs back and combine them into the original token order.

Mathematically:

$$Y_t = \sum_{e \in \mathcal{E}(t)} w_{t,e} f_e(X_t),$$

where:

- X_t is token t 's input,
- f_e is expert e ,
- $w_{t,e}$ is the router weight for token t and expert e ,
- Y_t is the combined output.

The dispatch phase sends X_t to selected experts. The combine phase returns $f_e(X_t)$ values and applies the weighted sum.

Why Device-Initiated Communication Matters for Experts

The router may run on the GPU. If routing decisions are produced on GPU, returning to the CPU to initiate communication is wasteful:

GPU router $\rightarrow$ CPU metadata extraction $\rightarrow$ CPU communication launch $\rightarrow$ GPU expert compute.

Device-initiated NVSHMEM avoids this:

GPU router → GPU put into expert mailbox → GPU expert compute.

This reduces latency and allows tighter fusion.

Portability Versus Specialization

The lecture discusses how some systems perform specialized low-level optimization for a specific network, such as InfiniBand. This can cut small synchronization overheads. However, it may reduce portability to other networks such as AWS EFA.

The trade-off is:

specialized path $\Rightarrow$ maximum performance on one environment,

public NVSHMEM API $\Rightarrow$ portable performance across multiple environments.

For production inference services that run across clouds and hardware generations, portability may be worth a small performance cost.

NCCL Roadmap and Convergence

The lecture emphasizes that NCCL and NVSHMEM are not static. NCCL is gaining features historically associated with NVSHMEM, including symmetric memory and device-side APIs.

The emerging direction is:

NCCL collectives+symmetric memory+device-side communication+NVLINK-aware pointer-like behavior.

This means the long-term choice may become less about raw feature availability and more about ecosystem, runtime semantics, and application structure.

Why NCCL Symmetric Memory Is Different from NVSHMEM Symmetric Heap

NVSHMEM has a world-oriented symmetric heap model. This is powerful but restrictive. Allocation is collective across the relevant PE set, and symmetry enables fast address translation.

The benefit is:

simple order-one address arithmetic.

The restriction is:

heterogeneous workflows are harder.

NCCL's communicator model can support more flexible subgroups:

`communicator1`, `communicator2`, overlapping or disjoint GPU groups.

This matters for AI systems where different parts of the application may use different parallelism groups:

`data parallel group`, `tensor parallel group`, `expert parallel group`, `pipeline group`.

A communicator-scoped symmetric memory model is more composable for heterogeneous distributed training and inference.

Common Misconceptions

Misconception: NCCL Is Just MPI for GPUs

NCCL is influenced by MPI, especially in its collective design, but it is not simply MPI for GPUs. NCCL removes or avoids many MPI features that would complicate GPU communication, such as arbitrary datatypes, wildcard matching, and multiple MPI ranks per GPU.

Misconception: MPI and NCCL Do the Same Thing

MPI and NCCL overlap in communication patterns, but their execution models differ. NCCL is built around CUDA streams and GPU-first communication. MPI is broader and more portable, but not always ideal for stream-ordered GPU workloads.

Misconception: One-Sided Communication Automatically Synchronizes

A put or get moves data, but it does not necessarily tell the remote side when it is safe to read or overwrite data. Synchronization must be added explicitly using barriers, flags, counters, or notification mechanisms.

Misconception: Remote Pointer Access Means Remote Memory Is Free

Remote pointer access simplifies code and can improve performance, but remote memory is still slower than local HBM. A remote NVLink load is not the same as a local HBM load.

The hierarchy remains:

$$B_{\text{HBM}} > B_{\text{NVLink}} > B_{\text{network}} > B_{\text{PCIe}} \text{ in many cases.}$$

Misconception: Fine-Grained Device Communication Is Always Better

Fine-grained communication can reduce latency when used correctly, but element-wise remote operations can be terrible if they create too many tiny messages. Warp-level or block-level aggregation is often needed.

Misconception: Users Should Manually Tune NCCL Protocols

NCCL exposes tuning controls, but the default algorithms are heavily optimized. Manual tuning can easily become non-portable and fragile.

Misconception: NVSHMEM Replaces NCCL

NVSHMEM and NCCL solve overlapping but distinct problems. NVSHMEM is excellent for one-sided, irregular, device-initiated, and remote-pointer-based algorithms. NCCL remains the default high-performance communication layer for many standard AI collectives and framework integrations.

Practical Takeaways

1. Use **NCCL** for standard AI collective patterns unless there is a clear reason not to.
2. Use **NVSHMEM** when the algorithm naturally needs one-sided communication, remote memory access, device-side communication, or mailbox-style routing.
3. Do not confuse data movement with synchronization. Two-sided communication often combines them; one-sided communication separates them.
4. In expert parallelism, the sender often knows the routing before the receiver does. This is a natural fit for one-sided put and notification.

5. Avoid elementwise remote communication unless the access pattern truly requires it and the hardware path supports it efficiently.
6. Prefer warp-level or block-level cooperative communication for GPU-initiated transfers.
7. Use `nvshmem_ptr` or equivalent remote pointer mechanisms to exploit NVLink domains when possible.
8. Always reason about topology. NVLink, PCIe, and inter-node networks have very different bandwidth and latency.
9. Trust NCCL’s default scheduling first. Profile before adding manual flow control.
10. For heterogeneous AI workflows, communicator-scoped memory and communication groups are important. NCCL’s direction toward symmetric memory may make these workflows easier.

Project or Experiment Ideas

Benchmark NCCL All-Reduce Across Message Sizes

Measure:

$$T(n) = \alpha + \frac{n}{B}$$

for NCCL all-reduce across a range of message sizes. Plot effective bandwidth:

$$B_{\text{eff}}(n) = \frac{n}{T(n)}.$$

Observe the transition from latency-bound to bandwidth-bound behavior.

Implement Matrix Transpose Variants

Implement the following on a multi-GPU system:

1. NCCL grouped send/receive all-to-all transpose,
2. NVSHMEM put-based transpose,
3. NVSHMEM get-based transpose,
4. NVSHMEM remote pointer transpose within an NVLink domain.

Measure effective bandwidth and compare against expected local HBM and NVLink bandwidths.

Study Elementwise Versus Block-Level NVSHMEM

Write two device-side kernels:

- one using elementwise remote get or put,
- one using block-level cooperative communication.

Compare performance as payload size grows. Confirm that aggregation improves latency amortization.

Build a Toy Expert Router

Implement a toy mixture-of-experts dispatch:

1. Generate token-to-expert assignments on GPU.
2. Use NVSHMEM put to place tokens into remote expert mailboxes.
3. Use counters or flags to notify arrival.
4. Run a simple expert function.
5. Combine results back to the source GPU.

The key experiment is to compare host-initiated communication versus device-initiated communication.

Profile with Nsight Systems and Nsight Compute

Use Nsight Systems to inspect:

- kernel launch count,
- stream ordering,
- NCCL communication phases,
- overlap between communication and compute.

Use Nsight Compute to inspect:

- memory throughput,
- occupancy,
- warp stalls,
- L2 behavior,
- memory coalescing.

Final Mental Model

NCCL and NVSHMEM represent two complementary ways to think about distributed GPU communication.

NCCL says:

Tell me the collective communication pattern, put it on a CUDA stream, and trust the runtime to choose the right algorithm for the topology.

NVSHMEM says:

Allocate symmetric memory, treat the distributed GPU system as a partitioned global address space, and directly move or access remote data when the algorithm needs fine-grained control.

The deepest distinction is synchronization. Two-sided communication makes synchronization part of the communication operation. One-sided communication gives the programmer direct data movement and requires explicit ordering.

For dense, regular, framework-level training, NCCL is usually the right first tool. For irregular routing, expert parallelism, remote mailboxes, GPU-initiated communication, and NVLink load/store access, NVSHMEM exposes capabilities that are difficult to express efficiently with traditional two-sided APIs.

The performance engineer’s job is to map the algorithm to the hardware hierarchy:

local HBM $\rightarrow$ NVLink $\rightarrow$ PCIe $\rightarrow$ inter-node network,

and to choose the programming model that avoids unnecessary synchronization, minimizes host involvement, coalesces communication, and keeps SMs focused on useful compute.